



兰州工业学院

实 验 报 告

(2023—2024 学年第 1 学期)

课程名称 《网络操作系统》

专业班级 专升本电信 23

学 号 ZB2305010116

姓 名 姜 勇

实验项目	实验五 Linux 命令编程实验		
实验时间	2023 年 11 月 10 日	实验地点	信息与通信工程基础实验室 (EDA 技术实验室)

一、实验内容

本实验涉及了多命令协作：命令行 CLI，命令行数据流，重定向的标准输入和输出，管道与命令等知识点。

通过本实验使学生掌握多命令协议的含义，掌握命令行 CLI 和数据流，掌握重定向的标准输入输出，熟悉管道及其相关命令。

对 Linux 的命令行进行重定向，管道统计、分流等操作；使用 Shell 脚本来进行程序的编译和安装。

二、实验器材（设备、元器件）

电脑、VMware Workstation Pro 虚拟机软件包、Centos 镜像包。

三、实验步骤及结果分析

第一部分

1.标准输入和输出

1) 在家目录下新建目录 twelve；（参考命令 mkdir）

图 1



图 1 使用 mkdir 命令在当前目录下创建了 twelve 目录

2) 进入目录 twelve (参考命令 cd)，目录 twelve 下建立 test1.txt，文件中输入” hello world” (使用 vi 编辑器)

图 2

```
dirk Python-3.11.0 test1.java test.s 模拟 下载
[jy@localhost ~]$ cd twelve
[jy@localhost twelve]$ touch test1.txt
[jy@localhost twelve]$ ls
test1.txt
[jy@localhost twelve]$ vim test1.txt
```

图 2 cd 命令进入到 twelve 目录中，使用 touch 命令新建了 test1.txt 文件

图 3

```
[jy@localhost twelve]$ cat test1.txt
hello world
[jy@localhost twelve]$
```

图 3 使用 vim 在 test1.txt 文件中输入 hello world，然后使用 cat 查看了文件

3) 在目录 twelve 下使用 cat 命令读取 test1.txt，并将结果输出到文件 test2.txt (参考命令 cat, >)，用 cat 查看 test2.txt 里的内容，将标准输出与标准错误至控制台

图 4

```
请按 ENTER 或其它命令继续
[jy@localhost twelve]$ cat test1.txt
hello world
[jy@localhost twelve]$ cat test1.txt >test2.txt
[jy@localhost twelve]$ ls
test1.txt  test2.txt
[jy@localhost twelve]$ cat test2.txt
hello world
[jy@localhost twelve]$
```

图 4 使用命令”cat test1.txt > test2.txt”将会将文件 test1.txt 的内容复制到 test2.txt，并且如果 test2.txt 不存在，则会创建该文件。如果 test2.txt 已经存在，那么原有的内容将会被覆盖。然后使用 cat 查看 test2.txt 里的内容

4) 复制之前 test-**<学号>.config** 的文件到目录 twelve 下，并改名为 wordAccount (参考命令 cp)；

图 5

```
[jy@localhost twelve]$ cp /home/jy/test-16.config .
[jy@localhost twelve]$ ls
test-16.config  test1.txt  test2.txt
[jy@localhost twelve]$ mv test-16.config wordAccount
[jy@localhost twelve]$ ls
test1.txt  test2.txt  wordAccount
[jy@localhost twelve]$
```

图 5 使用 cp 命令将 test-16.config 文件复制到当前目录，使用 mv 将复制过来的文件改名为 wordAccount

5) 用 cat 查看 test2 和 wordAccount 命令，并将正确结果和错误结果都填写到 Result 文件下面；

图 6

```
[jy@localhost twelve]$ cat test2.txt wordAccount > Result
[jy@localhost twelve]$ ls
Result test1.txt test2.txt wordAccount
[jy@localhost twelve]$ cat Result
hello world
#
#
# This file is used by the man-db package to configure the man and cat paths.
# It is also used to provide a manpath for those without one by examining
# their PATH environment variable. For details see the manpath(5) man page.
#
# Lines beginning with '#' are comments and are ignored. Any combination of
# tabs or spaces may be used as 'whitespace' separators.
#
# There are three mappings allowed in this file:
# -----
# MANDATORY_MANPATH                manpath_element
```

图 6 命令“cat test2.txt wordAccount > Result”将会把 test2.txt 文件和 wordAccount 文件的内容合并后输出到 Result 文件中，

6) 在 Result 文件内追加填入如下内容（参考命令 echo，“>>”）

This is all of it.

图 7

```
#-----
# Flags.
# NOCACHE keeps man from creating cat pages.
#NOCACHE
This is all of it.
[jy@localhost twelve]$
```

图 7 使用 echo “This is all of it.” 命令在文件内容的末尾添加了这个内容

7) 用 cat 查看 SumAccount 和 wordAccount 命令，并将正确结果和错误结果分别输出到 outCor 和 outErr 两个文件中，并进行检查。

图 8

```

# This is all of it.
[jy@localhost twelve]$ cat SumAccount >outCor 2>outErr
[jy@localhost twelve]$ cat wordAccount >>outCor 2>>outErr
[jy@localhost twelve]$ ls
outCor outErr Result test1.txt test2.txt wordAccount
[jy@localhost twelve]$ cat outCor
#
#
# This file is used by the man-db package to configure the man and cat paths.
# It is also used to provide a manpath for those without one by examining
# their PATH environment variable. For details see the manpath(5) man page.
#
# Lines beginning with '#' are comments and are ignored. Any combination of
# tabs or spaces may be used as 'whitespace' separators.
#
# There are three mappings allowed in this file:
# -----
# MANDATORY_MANPATH          manpath_element
# MANPATH_MAP                path_element    manpath_element
# MANDB_MAP                  global_manpath  [relative_catpath]
# -----
# every automatically generated MANPATH includes these fields
#

```

图 8 使用 cat 命令查看 SumAccount, wordAccount 文件的内容, 并将正确结果输出到 outCor 文件中, 将错误结果输出到 outErr 文件中。

图 9

```

# NOCACHE keeps man from creating cat pages.
#NOCACHE
[jy@localhost twelve]$ cat outErr
cat: SumAccount: 没有那个文件或目录
[jy@localhost twelve]$

```

图 9 使用 cat outErr 命令查看错误的结果

2.管道统计

1) 利用管道技术统计目录 twelve 下有多少个文件

图 10

```

jy@localhost:~
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
[jy@localhost ~]$ ls twelve
outCor outErr Result test1.txt test2.txt wordAccount
[jy@localhost ~]$ ls -l twelve | grep "^-" | wc -l
6
[jy@localhost ~]$

```

图 10 命令 `ls -l twelve`: 列出 `twelve` 目录下的文件和子目录的详细信息; 命令 `grep "^-"`: 从列出的详细信息中筛选出文件 (排除子目录), 即筛选以 “-” 开头的行; `wc -l`: 统计筛选出的行数, 即文件的数量。

2) 统计文件 `Result` 和 `wordAccount` 中的字数 (参考命令 `wc`)

图 11

```
[jy@localhost ~]$ wc -w Result wordAccount
wc: Result: 没有那个文件或目录
wc: wordAccount: 没有那个文件或目录
0 总用量
[jy@localhost ~]$ cd twelve
[jy@localhost twelve]$ wc -w Result wordAccount
 730 Result
 723 wordAccount
1453 总用量
[jy@localhost twelve]$
```

图 11 在 `twelve` 目录中使用 `wc -w Result wordAccount` 命令统计两个文件中的字数, `-w` 只统计单词数

3) 复制 `/etc/manpath.config` 文件到目录 `twelve` 下, 使用 `grep` 和 `wc` 统计该文件中 “MAN” 的个数

图 12

```
jy@localhost:~/twelve
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
[jy@localhost ~]$ cd twelve
[jy@localhost twelve]$ cp /etc/man_db.conf .
[jy@localhost twelve]$ ls
man_db.conf outCor outErr Result test1.txt test2.txt wordAccount
[jy@localhost twelve]$ grep -o -i "MAN" man_db.conf | wc -l
96
[jy@localhost twelve]$
```

图 12 先使用 `cp` 命令将 `/etc/manpath.config` 文件复制到 `twelve` 目录下, 然后使用 `grep` 和 `wc` 命令统计文件中 “MAN” 的数量

4) 统计当前登录用户的数量 (参考命令 `wc -l`)

图 13

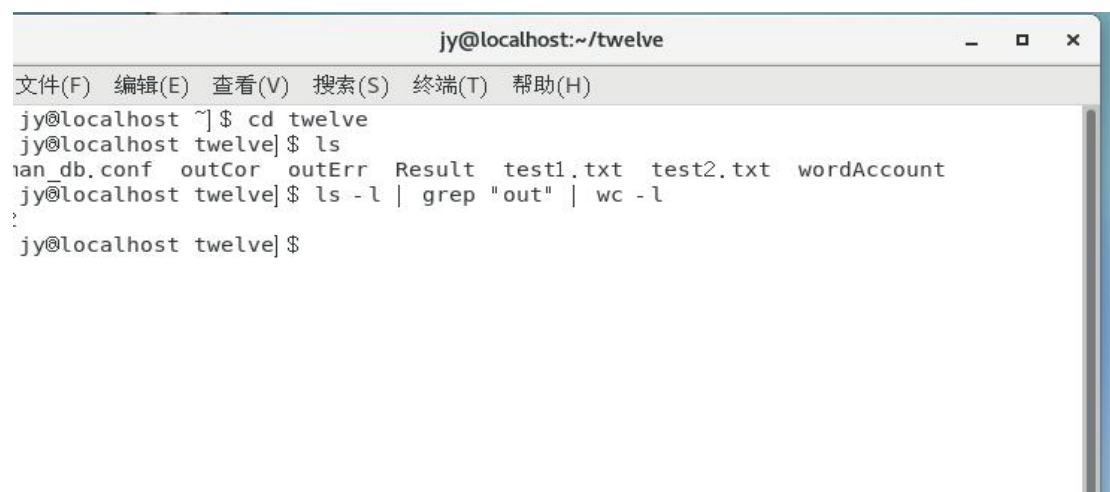


```
jy@localhost:~/twelve
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
[jy@localhost ~]$ cd twelve
[jy@localhost twelve]$ cp /etc/man_db.conf .
[jy@localhost twelve]$ ls
man_db.conf outCor outErr Result test1.txt test2.txt wordAccount
[jy@localhost twelve]$ grep -o -i "MAN" man_db.conf | wc -l
96
[jy@localhost twelve]$ who | wc -l
2
[jy@localhost twelve]$
```

图 13 使用 who 命令获取当前登录用户的信息，然后通过管道传递给 wc -l 命令进行行数统计，统计了当前登录用户的数量

5) 统计当前工作目录中共有多少个 out 开头的文件，

图 14



```
jy@localhost:~/twelve
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
jy@localhost ~]$ cd twelve
jy@localhost twelve]$ ls
man_db.conf outCor outErr Result test1.txt test2.txt wordAccount
jy@localhost twelve]$ ls -l | grep "out" | wc -l
2
jy@localhost twelve]$
```

图 14 命令 ls -l 列出当前目录下的所有文件和目录，并显示它们的详细信息，包括文件的权限、所有者、大小等。grep "out" 命令过滤出 ls -l 命令的输出结果中包含字符串 out 的行。wc -l 统计 grep "out" 命令的输出结果有多少行。

3.管道分流

1) 使用 tee 命令将 wordAccount 文件内容复制到 newAccount 中

图 15


```

[jy@localhost ~]$ cd twelve
[jy@localhost twelve]$ cat wordAccount | tee newAccount
#
# This file is used by the man-db package to configure the man and cat paths.
# It is also used to provide a manpath for those without one by examining
# their PATH environment variable. For details see the manpath(5) man page.
#
# Lines beginning with '#' are comments and are ignored. Any combination of
# tabs or spaces may be used as 'whitespace' separators.
#
# There are three mappings allowed in this file:
#-----
# MANDATORY_MANPATH      path_element      manpath_element
# MANPATH_MAP            path_element      manpath_element
# MANDB_MAP              global_manpath    [relative_catpath]
#-----
# every automatically generated MANPATH includes these fields
#
#MANDATORY_MANPATH      /usr/src/pvm3/man
#
MANDATORY_MANPATH      /usr/man

```

图 15 使用 cat 命令将文件 wordAccount 的内容将会在终端上显示，tee 命令在同一时间复制到文件 newaccount 中。

2) 使用 tee 命令将下述内容追加到 newAccount 文件后面

This is a new start.

图 16

```

This is a new start.
[jy@localhost twelve]$ cat -n newAccount | tail -10
 123 # the terminal actually being used. This should generally be within the
 124 # range set by MINCATWIDTH and MAXCATWIDTH.
 125 #
 126 #CATWIDTH      0
 127 #
 128 #-----
 129 # Flags.
 130 # NOCACHE keeps man from creating cat pages.
 131 #NOCACHE
 132 This is a new start.
[jy@localhost twelve]$ █

```

图 16 cat 命令将字符串 “This is a new start.” 输出到屏幕上，tee 命令将其追加到文件 newAccount.txt 的末尾。

第二部分

要求：以下所有的脚本需要通过以下三种方式进行运行：

方法一：chmod +x <filename>

./xxxxx.sh

方法二：bash <filename>

方法三：sh <filename>

1、编写脚本，根据用户输入的字符判断输入字符是 Y/y，还是 N/n，其他字母为退出：

```
#!/bin/bash

read -p "请输入一个字符:" input

if [ "$input" == "Y" ] || [ "$input" == "y" ];
then
    echo "您输入的是 Y/y"
elif [ "$input" == "N" ] || [ "$input" == "n" ];
then
    echo "您输入的是 N/n"
else
    echo "您输入的是其他字符，程序退出"。

    exit 0
fi
```

图 17

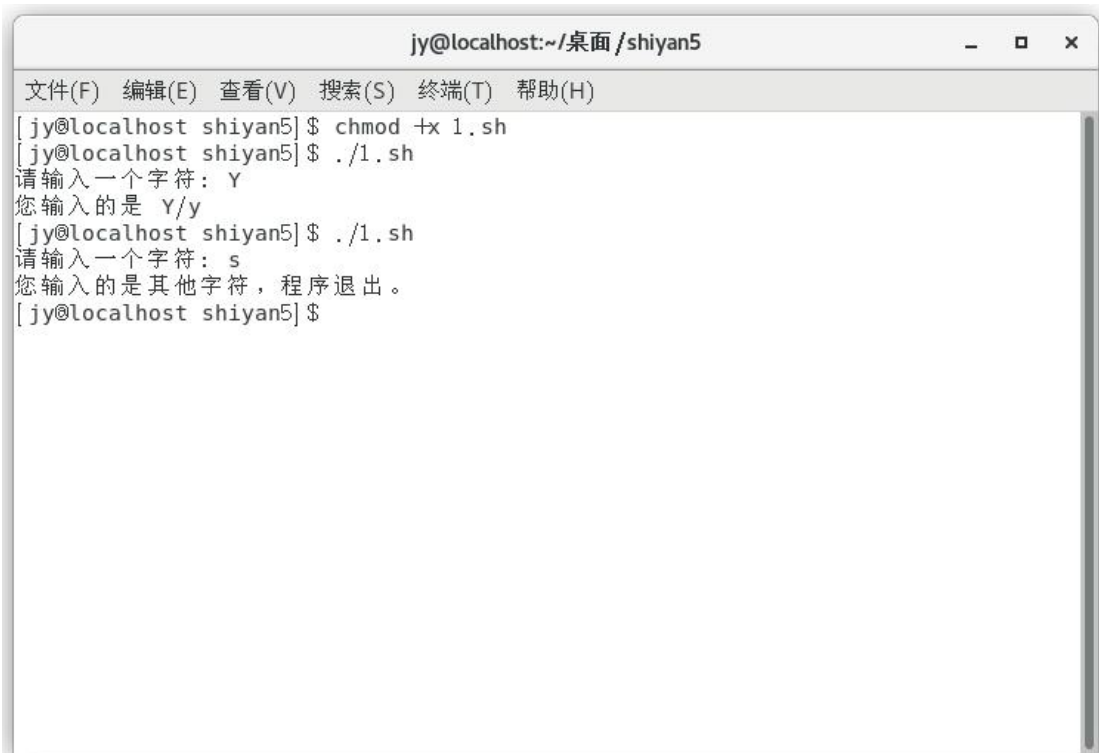
A terminal window titled 'jy@localhost:~/桌面/shiyan5' with standard window controls. The terminal shows the execution of a script. The user runs 'chmod +x 1.sh' and then './1.sh'. The script prompts '请输入一个字符: Y' and outputs '您输入的是 Y/y'. The user runs './1.sh' again, the script prompts '请输入一个字符: s' and outputs '您输入的是其他字符，程序退出。'. The prompt returns to '[jy@localhost shiyan5]\$'.

图 17 使用 chmod 命令给 1.sh 脚本给了一个可执行的权限，然后运行了此

脚本

2、从键盘录入一个数值，根据输入的值，判断是星期几，输入的数字如果超出 1-7 的范围就跳出程序 比如：输入：1 输出：星期 1

```
#!/bin/bash
read -p "请输入一个数字(1-7): " num
case $num in
1) echo "星期一"
;;
2)echo "星期二"
;;
3)echo "星期三"
;;
4)echo "星期四"
;;
5)echo "星期五"
;;
6)echo "星期六"
;;
7)echo "星期日"
;;
*)echo "输入的数字超出范围，程序退出。"
exit 0
;;
Esac
```

图 18

```
[jy@localhost shiyan5]$ bash 2.sh
请输入一个数字(1-7): 5
星期五
[jy@localhost shiyan5]$ bash 2.sh
请输入一个数字(1-7): 9
输入的数字超出范围，程序退出。
[jy@localhost shiyan5]$
```

图 18 使用 bash 命令运行了脚本，然后输入数字验证了星期是否合适

3、编写脚本,分别使用 while、for 完成 1 到 n 的数字相加

```
#!/bin/bash
read -p "请输入一个数字 n: " n
sum=0
for ((i=1; i<=n; i++));
do
    sum=$((sum+i))
done
echo "1 到 $n 的数字相加的结果为: $sum"
```

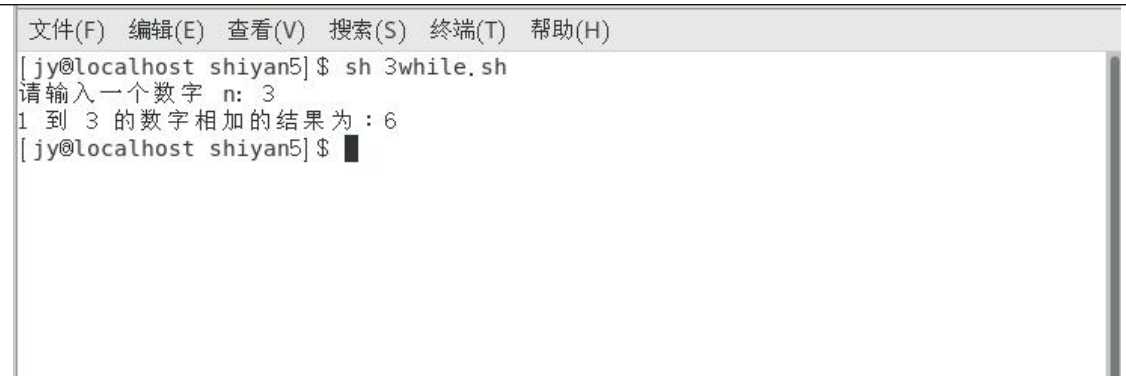
图 19

```
[jy@localhost shiyan5]$ sh 3for.sh
请输入一个数字 n: 5
1 到 5 的数字相加的结果为 : 15
[jy@localhost shiyan5]$
```

图 19 sh 命令运行脚本，输入了一个数字 5，代码 for 循环计算了 1-5 的和，然后输出

```
#!/bin/bash
read -p "请输入一个数字 n: " n
sum=0
i=1
while [ "$i" -le "$n" ]; do
    sum=$((sum+i))
    i=$((i+1))
done
echo "1 到 $n 的数字相加的结果为: $sum"
```

图 20



```
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
[jy@localhost shiyan5]$ sh 3while.sh
请输入一个数字 n: 3
1 到 3 的数字相加的结果为 : 6
[jy@localhost shiyan5]$
```

图 20 sh 命令运行脚本，此脚本是 while 循环计算了 1-3 的和然后输出

4、编写 shell 脚本，提示输入/etc 文件名，然后输出此目录内所有文件的权限，若可读输出 readable，若可写输出 writable、若可执行输出 executable

```
#!/bin/bash
read -p "请输入/etc 目录下的文件名: " dir
if [ ! -d "/etc/$dir" ];
then echo "输入的目录不存在，请重新运行脚本并输入正确的目录名。"
    exit 1
fi
for file in /etc/$dir/*;
do
    if [ -r "$file" ];
then readable="readable"
    else
        readable=""
    fi
    if [ -w "$file" ];
then writable="writable"
    else
        writable=""
    fi
    if [ -x "$file" ];
then executable="executable"
```

```

else
    executable=""
fi
echo "文件名: $file, 权限: $readable $writable $executable"
done

```

图 21

```

[jy@localhost shiyan5]$ bash 4.sh
请输入/etc目录下的文件名: man
输入的目录不存在, 请重新运行脚本并输入正确的目录名。
[jy@localhost shiyan5]$ bash 4.sh
请输入/etc目录下的文件名: abrt
文件名: /etc/abrt/abrt-action-save-package-data.conf, 权限: readable
文件名: /etc/abrt/abrt.conf, 权限: readable
文件名: /etc/abrt/gpg_keys.conf, 权限: readable
文件名: /etc/abrt/plugins, 权限: readable executable
[jy@localhost shiyan5]$

```

图 21 bash 运行脚本，此脚本是检查/etc 下的目录存不存在，存在的话输出此目录内所有文件的权限，可读输出 readable，可写输出 writable、可执行输出 executable

5、通过脚本的位置参数创建 Linux 系统账户及密码，执行脚本的第一个参数是用户名，执行脚本的第二个参数是密码

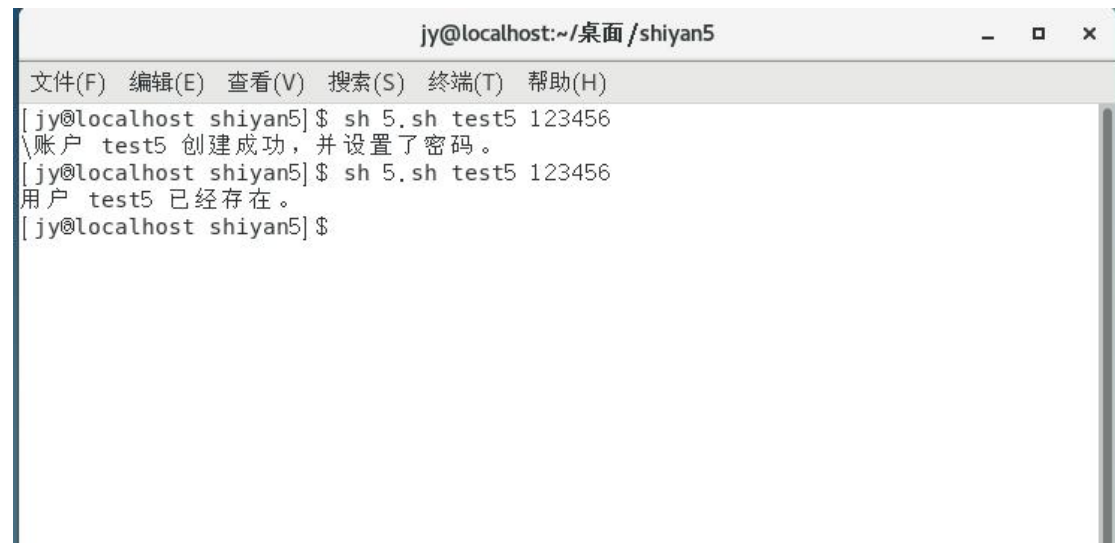
```

#!/bin/bash
if [ $# -ne 2 ];
then echo "请提供用户名和密码作为脚本的两个参数。"
    exit 1
fi
username=$1
password=$2
if id "$username" >/dev/null 2>&1;
then echo "用户 $username 已经存在。"
    exit 1
fi
sudo useradd -m $username
echo "$username:$password" | sudo chpasswd

```

echo "账户 \$username 创建成功，并设置了密码。"

图 22

A terminal window titled 'jy@localhost:~/桌面/shiyan5'. The window has a menu bar with '文件(F)', '编辑(E)', '查看(V)', '搜索(S)', '终端(T)', and '帮助(H)'. The terminal shows the following commands and output:

```
[jy@localhost shiyan5]$ sh 5.sh test5 123456
\账户 test5 创建成功，并设置了密码。
[jy@localhost shiyan5]$ sh 5.sh test5 123456
用户 test5 已经存在。
[jy@localhost shiyan5]$
```

图 22 sh 命令运行了脚本，此脚本是创建用户并设置密码，在运行脚本时在后面跟上要创建的用户名和密码两个参数

6、脚本生成一个 100 以内的随机数，提示用户猜数字，根据用户的输入，提示用户猜对了，猜小了或猜大了，直至用户猜对脚本结束。（#RANDOM 为系统自带的系统变量，值为 0 - 32767 的随机数，#使用取余算法将随机数变为 1 - 100 的随机数）

```
#!/bin/bash
```

```
# 生成 1-100 的随机数 andom_number=$((RANDOM % 100 + 1))
```

```
echo "猜数字游戏开始！"
```

```
while true; do
```

```
    read -p "请猜一个 1-100 的数字：" guess
```

```
    if ! [[ "$guess" =~ ^[0-9]+$ ]];
```

```
then echo "请输入有效的数字！"
```

```
        continue
```

```
    fi
```

```
    if [ "$guess" -lt "$random_number" ];
```

```
then echo "猜小了！再试一次。"
```

```
        elif [ "$guess" -gt "$random_number" ];
```

```
then echo "猜大了！再试一次。"
```

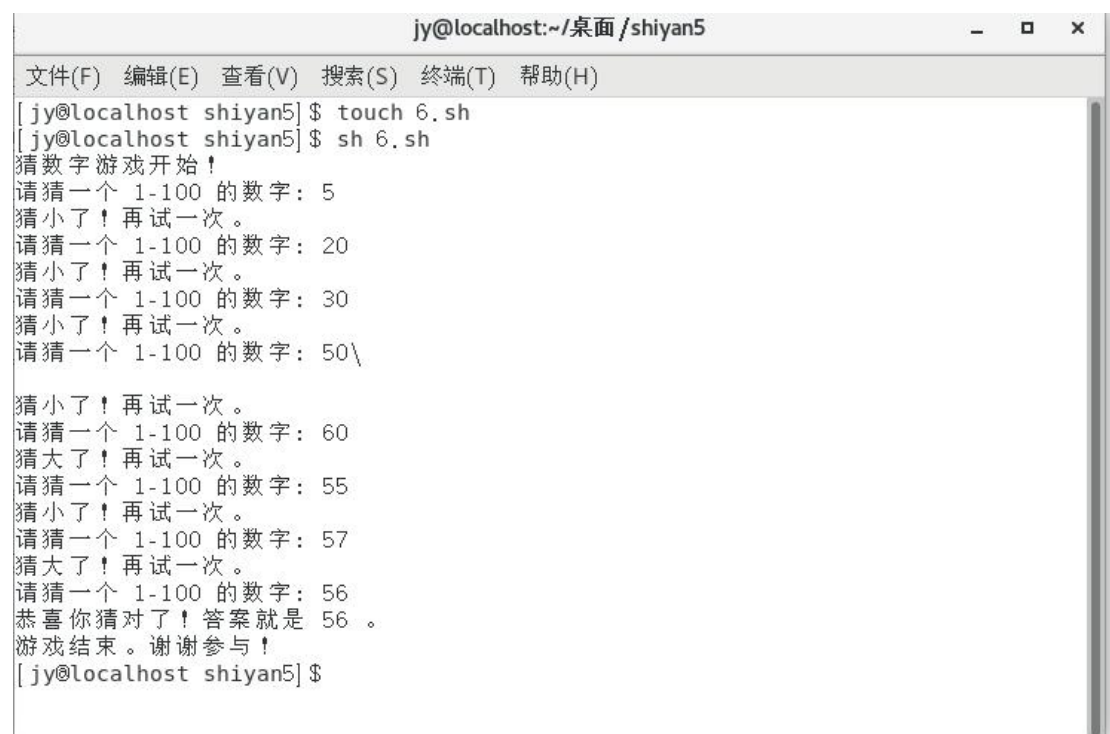


```

else
    echo "恭喜你猜对了！答案就是 $random_number 。"
    break
fi
done
echo "游戏结束。谢谢参与！"

```

图 23



```

jy@localhost:~/桌面/shiyan5
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
[jy@localhost shiyan5]$ touch 6.sh
[jy@localhost shiyan5]$ sh 6.sh
猜数字游戏开始！
请猜一个 1-100 的数字：5
猜小了！再试一次。
请猜一个 1-100 的数字：20
猜小了！再试一次。
请猜一个 1-100 的数字：30
猜小了！再试一次。
请猜一个 1-100 的数字：50\
猜小了！再试一次。
请猜一个 1-100 的数字：60
猜大了！再试一次。
请猜一个 1-100 的数字：55
猜小了！再试一次。
请猜一个 1-100 的数字：57
猜大了！再试一次。
请猜一个 1-100 的数字：56
恭喜你猜对了！答案就是 56 。
游戏结束。谢谢参与！
[jy@localhost shiyan5]$

```

图 23 sh 运行脚本，此脚本生成一个 100 以内的随机数，用户猜数字，直至用户猜对脚本结束

7、编写一个 shell 脚本，完成以下功能。

- (1)显示文字 “Waiting for a while…”。
- (2)长格式显示当前目录下面的文件和目录，并输出重定向到/home/fle.txt 文件。
- (3)定义一个变量，名为 s，初始值 “Hello”。
- (4)使该变量输出重定向到/home/string.txt 文件。

```

#!/bin/bash
echo "Waiting for a while..."

```

```
ls -l > /home/file.txt
s="Hello"
echo $s > /home/string.txt
```

图 24

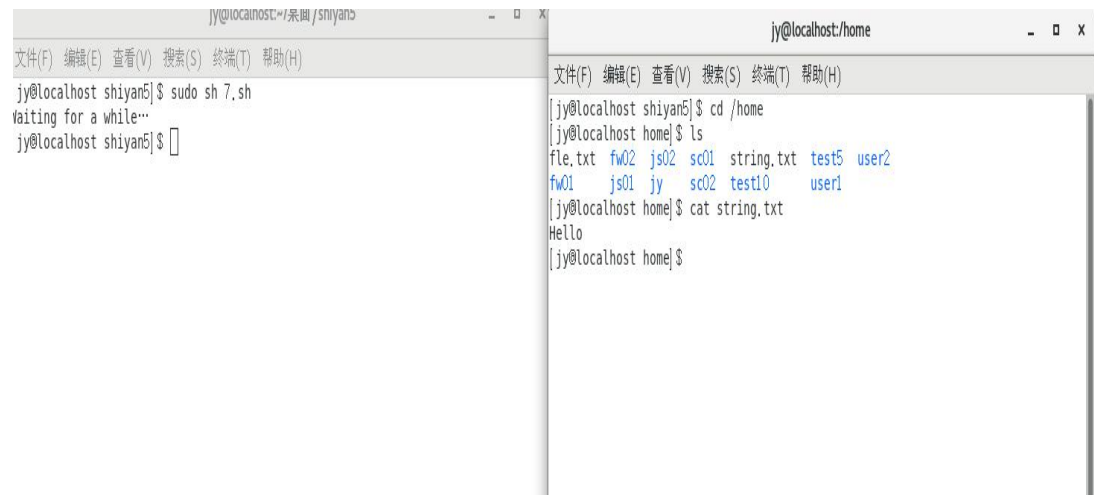


图 24 此脚本是显示文字“Waiting for a while...”，显示当前目录下面的文件和目录，并输出重定向到/home/file.txt 文件。定义一个变量，名为 s，初始值“Hello”。使该变量输出重定向到/home/string.txt 文件。

扩展题：

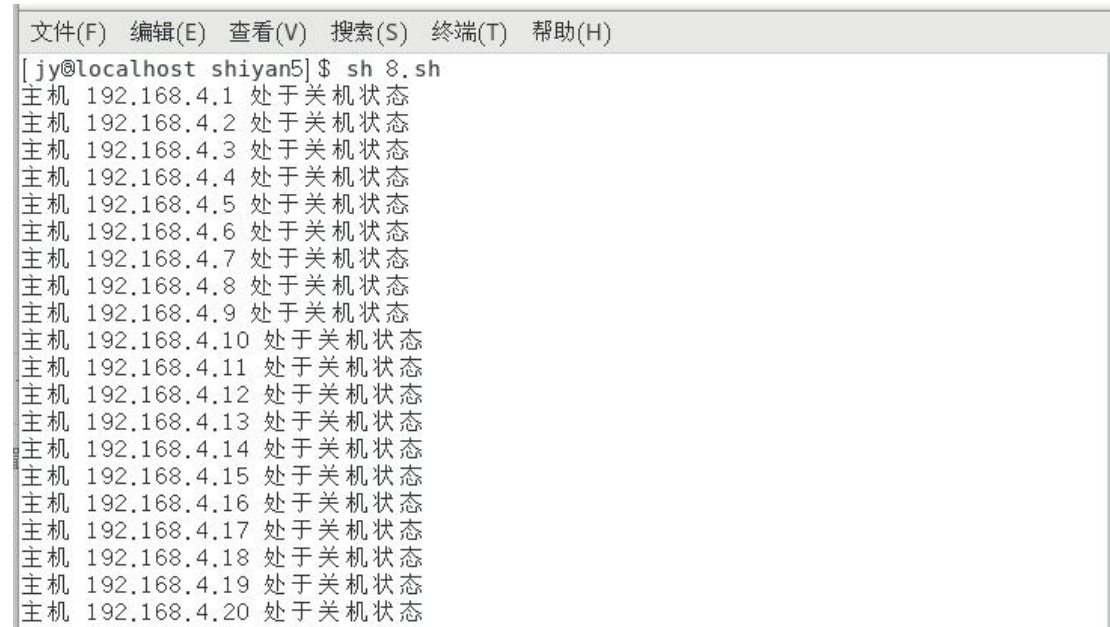
8、编写脚本测试 192.168.4.0/24 整个网段中哪些主机处于开机状态，哪些主机处于关机状态

```
#!/bin/bash
subnet="192.168.4."
mask="255.255.255.0"
for i in $(seq 1 24); do
    ip=$subnet$i
    response=$(ping -c 1 -W 1 $ip 2> /dev/null)
    if [ $? -eq 0 ]; then
        echo "主机 $ip 处于开机状态"
    else
        echo "主机 $ip 处于关机状态"

```

```
fi
done
```

图 25



```
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
[jy@localhost shiyan5]$ sh 8.sh
主机 192.168.4.1 处于关机状态
主机 192.168.4.2 处于关机状态
主机 192.168.4.3 处于关机状态
主机 192.168.4.4 处于关机状态
主机 192.168.4.5 处于关机状态
主机 192.168.4.6 处于关机状态
主机 192.168.4.7 处于关机状态
主机 192.168.4.8 处于关机状态
主机 192.168.4.9 处于关机状态
主机 192.168.4.10 处于关机状态
主机 192.168.4.11 处于关机状态
主机 192.168.4.12 处于关机状态
主机 192.168.4.13 处于关机状态
主机 192.168.4.14 处于关机状态
主机 192.168.4.15 处于关机状态
主机 192.168.4.16 处于关机状态
主机 192.168.4.17 处于关机状态
主机 192.168.4.18 处于关机状态
主机 192.168.4.19 处于关机状态
主机 192.168.4.20 处于关机状态
```

图 25 此脚本测试 192.168.4.0/24 整个网段中哪些主机处于开机状态，哪些主机处于关机状态

9、编写批量修改扩展名脚本，批量将《实验五 Linux 命令编程实验（一）》中目录 twelve 下后缀名为 out 的文件修改为后缀名为 txt 文件（使用循环，通过 ls 找到相关文件，mv 进行重新名）

```
#!/bin/bash
directory="/home/jy/twelve"
old_extension=".out"
new_extension=".txt"
echo "批量修改文件后缀名为 .txt: "
cd "$directory" || exit
for file in *"$old_extension"; do
    if [ -f "$file" ]; then
        filename="${file%$old_extension}"
        new_file="$filename$new_extension"
        mv "$file" "$new_file"
    fi
done
```

```

        echo "已修改文件名: $file -> $new_file"
    fi
done
echo "批量修改后缀名完成。"

```

图 26



图 26 此脚本批量修改扩展名 out 为 txt

10、设计一个 Shell 程序，添加一个新组为 class1，然后添加属于这个组的 30 个用户，用户名的形式为 stdxx，其中 xx 为 1-30。

```

#!/bin/bash
group_name="class1"
echo "添加新组 '$group_name': "
groupadd "$group_name"
echo "添加属于组 '$group_name' 的用户: "
for i in {1..30}; do
    username="std$(printf "%02d" $i)"
    password=$(openssl rand -base64 12)
    useradd -m -s /bin/bash -g "$group_name" "$username"
done

```

```

echo "$username:$password" | chpasswd

echo "已创建用户 '$username'，密码为 '$password'"

done

echo "用户添加完成。"

```

图 27



```

jy@localhost shiyan5]$ sudo 10.sh
sudo: 10.sh: 找不到命令
jy@localhost shiyan5]$ sudo sh 10.sh
添加新组 'class1':
添加属于组 'class1' 的用户:
已创建用户 'std01', 密码为 'acHrQY3NbeL50/s3'
已创建用户 'std02', 密码为 'rtvaCT2Z44EuT0ug'
已创建用户 'std03', 密码为 'kmPOLKp30xMpFHqw'
已创建用户 'std04', 密码为 'DfLYLYG7BfMf3F3K'
已创建用户 'std05', 密码为 '26ONMmvscy7XxD7f'
已创建用户 'std06', 密码为 'Iu1fkWgffw/ha9un'
已创建用户 'std07', 密码为 'hqS+ffZBBJ8zBxq+'
已创建用户 'std08', 密码为 'J0XZ7ncUI/pINsRT'
已创建用户 'std09', 密码为 '53ti3wKWQuggp5CY'
已创建用户 'std10', 密码为 'Uwob3HNqPgDahnS0'
已创建用户 'std11', 密码为 'SaxGCvar8LPHRpmd'
已创建用户 'std12', 密码为 '83IkAeryYrDqckh2'
已创建用户 'std13', 密码为 'Qu6zW00cKDvuVdcl'
已创建用户 'std14', 密码为 'npaNEK06XKGHRaGf'
已创建用户 'std15', 密码为 'VrPHiwiGd8Gabo0l'
已创建用户 'std16', 密码为 '+Hbdo480saSNKBLH'
已创建用户 'std17', 密码为 'cxsTGFf2iqhnSOV'
已创建用户 'std18', 密码为 'rRu/qcUFC3g52Gj+'
已创建用户 'std19', 密码为 '9lgdYyRYcEmRh/Ao'

```

图 27 此脚本创建了一个用户组，然后创建了 30 个用户添加到了这个组中

11、设计一个 Shell 程序，删除 9 题创建的所有用户和组，连同用户目录一并删除。

```

#!/bin/bash

group_name="class1"

for i in {1..30}; do

    username="std$(printf "%02d" $i)"

    userdel -r "$username"

    echo "已删除用户 '$username' 及其用户目录"

done

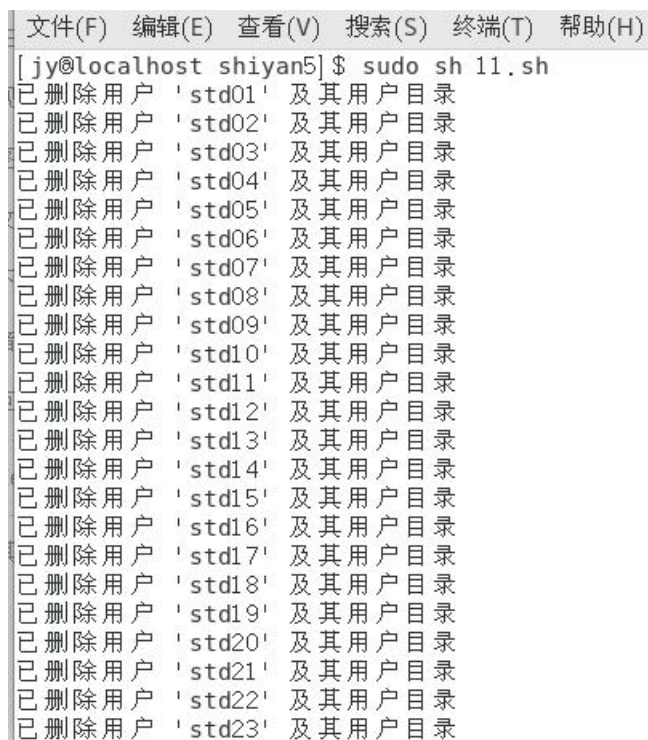
groupdel "$group_name"

echo "已删除组 '$group_name'"

echo "用户和组删除成功。"

```

图 28



```
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
[jy@localhost shiyan5]$ sudo sh 11.sh
已删除用户 'std01' 及其用户目录
已删除用户 'std02' 及其用户目录
已删除用户 'std03' 及其用户目录
已删除用户 'std04' 及其用户目录
已删除用户 'std05' 及其用户目录
已删除用户 'std06' 及其用户目录
已删除用户 'std07' 及其用户目录
已删除用户 'std08' 及其用户目录
已删除用户 'std09' 及其用户目录
已删除用户 'std10' 及其用户目录
已删除用户 'std11' 及其用户目录
已删除用户 'std12' 及其用户目录
已删除用户 'std13' 及其用户目录
已删除用户 'std14' 及其用户目录
已删除用户 'std15' 及其用户目录
已删除用户 'std16' 及其用户目录
已删除用户 'std17' 及其用户目录
已删除用户 'std18' 及其用户目录
已删除用户 'std19' 及其用户目录
已删除用户 'std20' 及其用户目录
已删除用户 'std21' 及其用户目录
已删除用户 'std22' 及其用户目录
已删除用户 'std23' 及其用户目录
```

图 28 此脚本删除了上题创建的所有用户和组，连同用户目录一并删除。

四、实验总结

通过本次实验，我了解了一些文件处理和文本处理命令。例如，**cat** 命令可以显示文件内容，**grep** 命令可以在文件或输出中搜索指定的字符串，**wc** 命令可以计算文件或输出的行数、字数和字符数。这些命令使我能够在文件中查找、过滤和统计信息。另外，我还学习了如何使用管道和重定向来处理命令的输入和输出。使用管道可以将一个命令的输出作为另一个命令的输入，这样可以将多个命令组合在一起实现更复杂的操作。而重定向则可以将命令的输出重定向到文件中，或者从文件中读取输入。这些技巧对于自动化任务和批量处理非常有用。

学会了 **sh** 脚本，懂得了 4 中运行 **sh** 脚本的方法，1.赋予脚本执行权限：**chmod +x xx.sh**，在终端中运行命令 **./script.sh**，执行脚本。2.在终端中运行命令 **bash xx.sh**。3.在终端中运行命令 **sh xx.sh**。4.在终端中运行命令 **source xx.sh**。

实验成绩评定表

序号	考核项目	分值分布	成绩
1	出勤与纪律情况	10	
2	实验任务完成情况	40	
3	实验报告质量	50	
总分			
指导教师签字			

备注：考核项目及分值分布应依据教学大纲确定。